

Exercise 5

Students

This report was prepared by the following students:

- **Student 1:** Abdulghani Almasri
- **Student 2:** Alvaro Garmendia

1. Implementation of Ring Allreduce

Goal

Every process starts with its own array of `count` floats. After the operation, every process holds, at each index, the **sum of that index across all processes**. This is a sum- `Allreduce` .

Following the lecture, the operation is built from two ring phases:

1. **Ring Reduce-Scatter** — after this phase, each process owns the *complete* sum of exactly one chunk of the array.
2. **Ring Allgather** — circulates those completed chunks around the ring so every process ends up with all of them.

Both phases use the same ring topology: each process only ever sends to its **successor** and receives from its **predecessor**.

```
int next = mod(rank + 1, size); // successor - we always send here
int prev = mod(rank - 1, size); // predecessor - we always receive here
```

Why a custom modulo

The chunk and neighbour indices are computed with expressions like `r - i` , which go negative. In C, `%` is the *remainder* operator and keeps the sign of the left operand, so `(-1) % 8` is `-1` , not `7` — that would index outside the array. A small wrapper gives a true modulo:

```
static inline int mod(int a, int b) {
    return ((a % b) + b) % b;
}
```

Every neighbour and chunk index in the algorithm goes through `mod` .

Splitting the array into chunks

The array is divided into exactly `size` chunks, kept **as equal as possible**: when `count` is not divisible by `size` , the first `count % size` chunks get one extra element. Offsets and lengths are precomputed once so the rest of the code can address any chunk directly:

```
int base = count / size;
int rem = count % size;
for (int c = 0; c < size; ++c) {
    len[c] = base + (c < rem ? 1 : 0);
    offset[c] = /* running sum of previous lengths */;
}
```

This balances the per-step message sizes, which matters because the ring is bandwidth-bound.

Phase 1 — Ring Reduce-Scatter

The phase runs `size - 1` iterations. In iteration `i` , process `r` :

- **sends** chunk `(r - i) mod size` to its successor,
- **receives** chunk `(r - (i+1)) mod size` from its predecessor into a temp buffer,
- **adds** the received values onto its own copy of that chunk.

`MPI_Sendrecv` performs the send and receive together, so the ring never deadlocks regardless of message size:

```
MPI_Sendrecv(recvbuf + offset[send_idx], len[send_idx], MPI_FLOAT, next, 0,
             tmp, len[recv_idx], MPI_FLOAT, prev, 0,
             comm, MPI_STATUS_IGNORE);
// then: recvbuf[recv_chunk] += tmp
```

Each chunk accumulates one more process's contribution per iteration. After the last iteration, the chunk `(r + 1) mod size` on process `r` holds the **full sum** of that chunk. The other chunks hold partial sums that will be replaced in phase 2.

Phase 2 — Ring Allgather

This phase also runs `size - 1` iterations. In iteration `i`, process `r`:

- **sends** chunk `(r + 1 - i) mod size` to its successor,
- **receives** chunk `(r - i) mod size` from its predecessor,
- **overwrites** its local chunk with what it receives (no addition this time).

The first thing each process sends is its fully-summed chunk from phase 1; each step then forwards a completed chunk one hop further around the ring. Because the send and receive chunks are always adjacent (and therefore never overlap), the data can be received straight into `recvbuf`:

```
MPI_Sendrecv(recvbuf + offset[send_idx], len[send_idx], MPI_FLOAT, next, 1,
             recvbuf + offset[recv_idx], len[recv_idx], MPI_FLOAT, prev, 1,
             comm, MPI_STATUS_IGNORE);
```

After `size - 1` iterations, every process has every completed chunk — i.e. the full summed array.

Why the ring, and what it costs

Each rank sends and receives roughly $2 \cdot (\text{size} - 1) / \text{size} \cdot \text{count}$ floats in total, **independent of a single root**. No process becomes a bottleneck and every link stays busy, which is why the ring is the bandwidth-efficient choice on uniform networks.

Filling the input with rank-specific data

Before the algorithm runs, every process fills its own input array. To make the test meaningful, each rank writes **different** values, so a correct result can only come from actually combining data across all ranks:

```
for (int i = 0; i < N; ++i)
    input[i] = (float) (rank + 1) * 0.5f + (float) (i % 17);
```

The value at index `i` has two parts:

- `(rank + 1) * 0.5` is a **per-rank offset** — `0.5` on rank 0, `1.0` on rank 1, `1.5` on rank 2, and so on. Because it depends on `rank`, every process contributes a distinct amount at the same index. (Using `rank + 1` rather than `rank` keeps rank 0 from contributing zero everywhere.)
- `i % 17` is a **per-index pattern** that repeats every 17 elements. It varies the values along the array so a chunk isn't just a constant, which would hide bugs that misplace or drop elements.

Each process runs this loop independently — there is no communication here. With `size` ranks, the expected sum at index `i` is therefore

```
sum_over_ranks( (rank + 1) * 0.5 ) + size * (i % 17)
= 0.5 * (1 + 2 + ... + size) + size * (i % 17)
```

which is a known, closed-form value. That makes the result easy to reason about and is exactly what the correctness check compares against (via MPI's native `MPI_Allreduce` on the same input).

Correctness check

The program compares the result against MPI's native `MPI_Allreduce`. The comparison uses a **relative** epsilon rather than an absolute one:

```
double denom = (fabs(b) > 1e-30) ? fabs(b) : 1.0;
if (fabs(a - b) / denom > rel_eps) local_errors++; // rel_eps = 1e-5
```

This matters because floating-point addition is not associative: the ring sums values in a different order than MPI does, so tiny differences are expected. With larger inputs the spacing between representable floats grows beyond 1, so an absolute tolerance would raise false alarms — a relative tolerance does not.

Sample run

A test on a single node with 8 processes and a 1000-element array:

```
N=1000 procs=8 -> CORRECT (0 mismatches)
ring_allreduce: 0.000251 s MPI_Allreduce: 0.000060 s
```

The result matches the native implementation exactly (no mismatches). At this tiny size native is faster, which is expected: 1000 floats is latency-dominated, and the hand-written ring pays $2 \cdot (\text{size} - 1)$ communication steps where MPI uses a latency-optimised algorithm for such small messages. The ring's advantage shows up for large, bandwidth-bound messages — and, as the scaling experiments show, when the processes are spread across separate nodes.

2. Ring Allreduce – scaling problem size

Setup: 8 processes on octane, problem size swept in powers of two from 2^{16} to 2^{22} floats. Each value is the average over 5 runs of the timed collective only (initialization excluded), reported as the maximum over all ranks. Two process layouts are compared: all 8 processes on

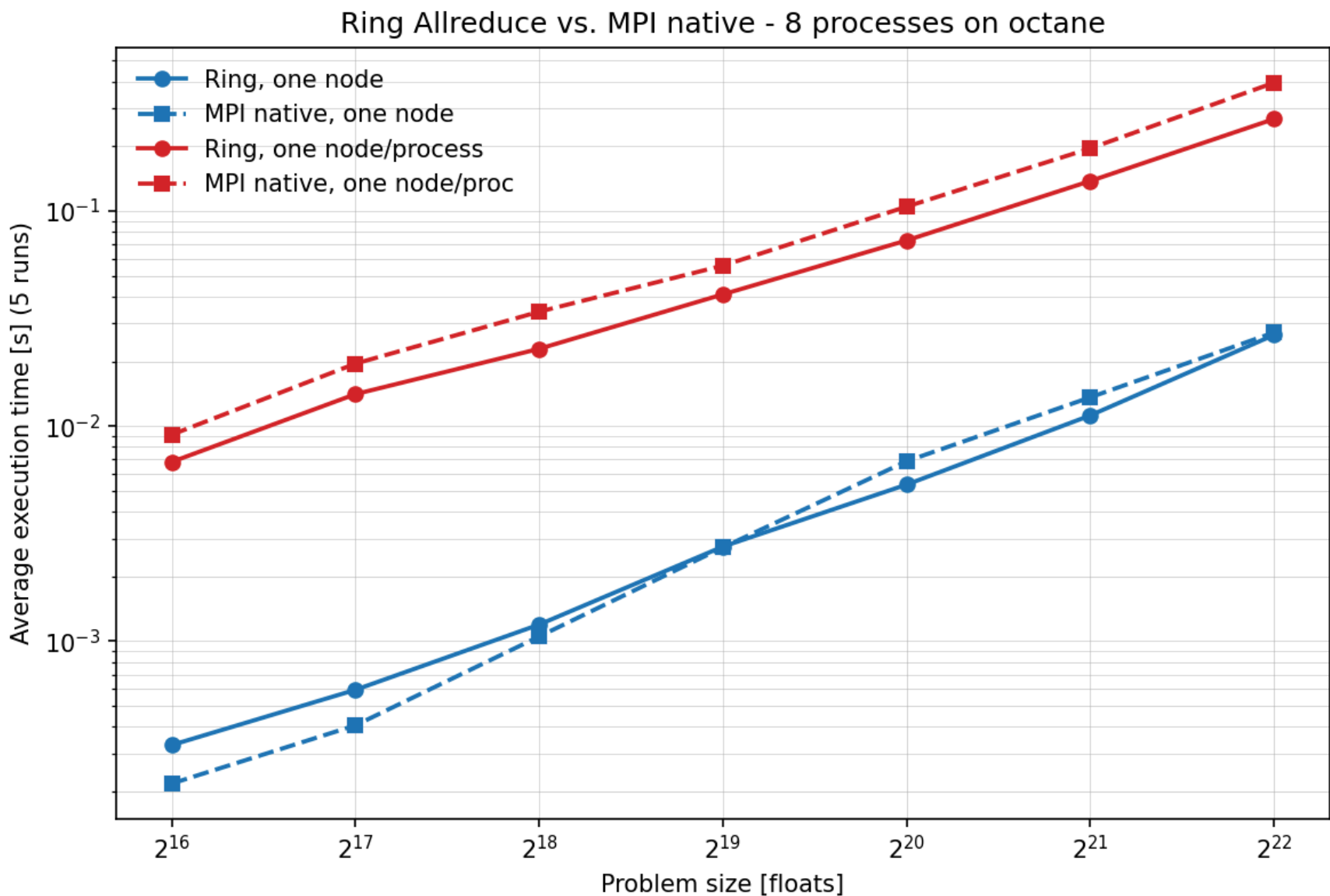
one node, and **one process per node** across 8 nodes. All runs verified correct against MPI's native `MPI_Allreduce` .

Results

Table 1: Execution time t_{ex} [s]

Problem Size	Ring Allreduce, One Node [s]	MPI Native, One Node [s]	Ring Allreduce, One Node per Process [s]	MPI Native, One Node per Process [s]
2^{16}	0.000329	0.000218	0.006816	0.009123
2^{17}	0.000595	0.000407	0.014079	0.019520
2^{18}	0.001196	0.001052	0.022869	0.034023
2^{19}	0.002754	0.002730	0.040955	0.055712
2^{20}	0.005348	0.006865	0.072968	0.104706
2^{21}	0.011215	0.013611	0.137815	0.195430
2^{22}	0.026535	0.027239	0.267575	0.394606

Plot



Solid lines are the ring implementation, dashed lines MPI native; blue is the one-node layout, red is one-process-per-node. Both axes are logarithmic.

Discussion

Two regimes. The one-node and inter-node curves sit an order of magnitude apart (≈ 0.027 s vs ≈ 0.268 s at 2^{22} , $\sim 10\times$ slower across nodes) — shared memory within a node versus the cluster network between them. At 2^{22} each rank moves about $2 \cdot (\text{size}-1) / \text{size} \cdot \text{count} \approx 29$ MB, giving an effective ~ 110 MB/s across nodes (Gigabit Ethernet) and ~ 1.1 GB/s within a node.

Bandwidth-bound. On the log-log plot the curves are straight with slope ≈ 1 at large sizes, so time grows linearly with message size.

One node: native wins on small messages ($\sim 1.5\times$ at 2^{16}) and the ring wins on large ones ($1.2\text{--}1.3\times$ at $2^{20}\text{--}2^{21}$), crossing over near 2^{19} . This is the latency–bandwidth tradeoff: small messages are latency-bound and the ring pays $2 \cdot (\text{size}-1)$ steps, while large messages are bandwidth-bound where the ring's optimal data movement wins.

One process per node: the ring beats native at every size by a steady $1.3\text{--}1.5\times$, the expected result. Each rank moves only $\sim 2 \cdot (\text{size}-1) / \text{size} \cdot \text{count}$ of data, never funneling through a root, and all links stay busy — near-optimal on a bandwidth-limited fabric. The roughly constant ratio confirms both are bandwidth-bound and differ only by a constant factor in bytes moved.

All configurations produced numerically correct results.

3. Ring Allreduce - scaling process count

Setup: fixed problem size $N = 1,000,000$ floats, process count swept from 2 to 8. Each value is the average over 5 runs of the timed collective only (initialization excluded), reported as the maximum over all ranks. Two layouts are compared: all processes on **one node**, and **one process per node**. All runs verified correct against MPI's native `MPI_Allreduce`.

Results

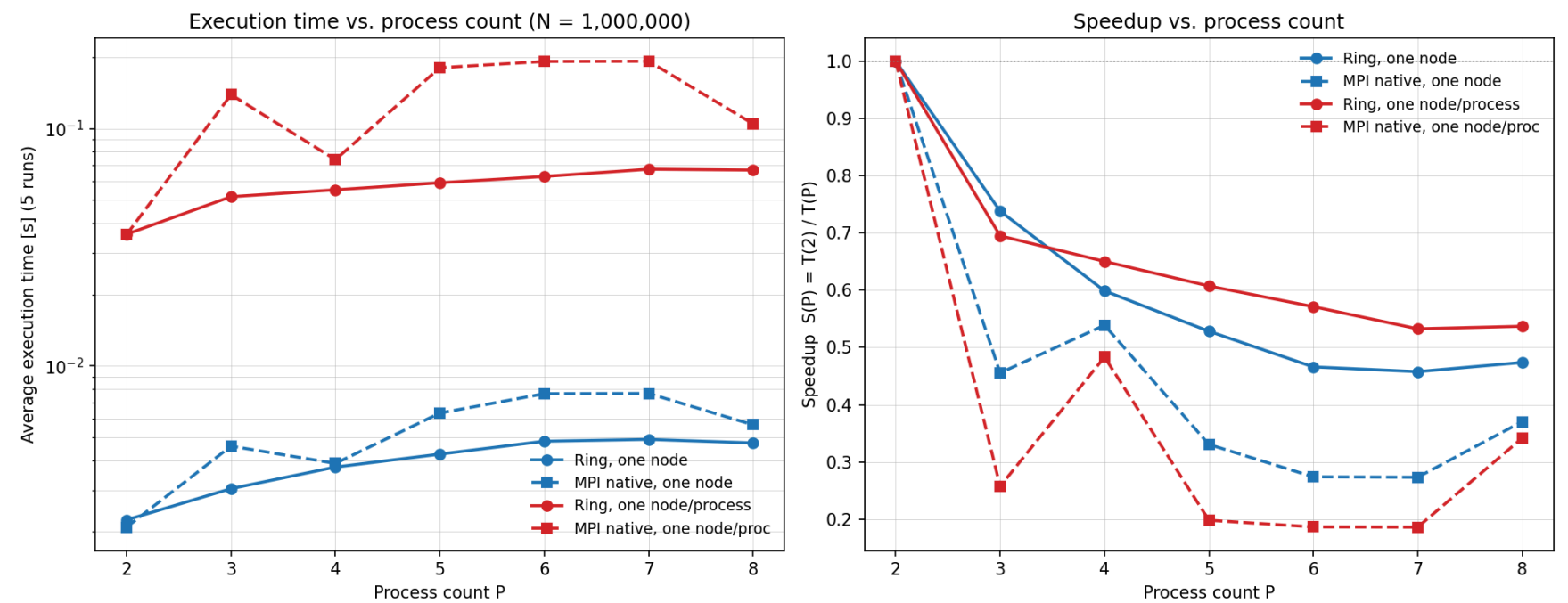
Table 1: Execution time t_{ex} [s]

Process Count	Ring Allreduce, One Node [s]	MPI Native, One Node [s]	Ring Allreduce, One Node per Process [s]	MPI Native, One Node per Process [s]
2	0.002222	0.002132	0.035899	0.035846
3	0.003171	0.004696	0.051545	0.138771
4	0.003821	0.003960	0.055262	0.080137
5	0.004161	0.006780	0.061658	0.180778
6	0.004864	0.008030	0.062308	0.190028
7	0.005099	0.007728	0.064821	0.189457
8	0.004866	0.006123	0.163982	0.106872

Table 2: Speedup $S(P) = T(2) / T(P)$, per column

Process Count	Ring Allreduce, One Node	MPI Native, One Node	Ring Allreduce, One Node per Process	MPI Native, One Node per Process
2	1.000	1.000	1.000	1.000
3	0.701	0.454	0.696	0.258
4	0.582	0.539	0.650	0.447
5	0.534	0.315	0.582	0.198
6	0.457	0.266	0.576	0.189
7	0.436	0.276	0.554	0.189
8	0.457	0.348	0.219	0.335

Plot



Left: execution time (log axis). Right: speedup $S(P) = T(2)/T(P)$. Solid = ring, dashed = native; blue = one node, red = one node per process.

Discussion

Speedup decreases with P — and that is expected. N is fixed, and in an allreduce every process holds the *full* array regardless of P , so nothing is divided among more workers. Adding processes only adds communication: the ring cost is $2 \cdot (P-1)/P \cdot N$ of data (approaching a constant $2N$) plus $2 \cdot (P-1)$ latency steps (growing linearly with P). So $T(P)$ rises with P and $S(P) = T(2)/T(P)$ falls. There is no single-process baseline — allreduce on one rank does no communication — so the curve is normalised to $P = 2$, where it starts at 1.0.

Two regimes. Inter-node times are $\sim 10\text{--}30\times$ the intra-node ones (network vs. shared memory), consistent with Exercise 5.2. $P = 2$ is a near-tie in both layouts: with two ranks the ring is a single exchange, identical to what native does.

The ring beats native almost everywhere. Within a node the ring is faster for $P \geq 3$ ($1.3\text{--}1.7\times$, $P = 4$ a near-tie). Across nodes the ring wins for $P = 3\text{--}7$ by a large margin ($1.5\text{--}3.0\times$), the result the exercise expects, because its balanced, bandwidth-optimal traffic suits the bandwidth-limited fabric while native moves more data and/or leaves links idle. The native inter-node column is also very erratic (0.139, 0.080, 0.181, 0.190, ...), reflecting MPI switching algorithms by communicator size on top of contention from the shared (non-exclusive) nodes.

One outlier: $P = 8$ inter-node ring (0.164 s). This single point is $\sim 2.5\times$ above the smooth trend the other sizes follow (~ 0.07 s) and is the only case where the ring loses inter-node. It is almost certainly transient network contention: the per-node layout runs without `--exclusive`, $P = 8$ exposes all eight nodes at once, and a single slow rep is not averaged out over only 5 runs. Re-running this point (at a quieter time, or with more repetitions / the median) should bring it back to ~ 0.07 s, restoring the ring's advantage across the whole range.

All configurations produced numerically correct results.

4. Willingness to present

- Ring Allreduce - Implementation (Section: 5.1)
- Ring Allreduce - Scaling problem size (Section: 5.2)
- Ring Allreduce - Scaling process count (Section: 5.3)